You can use a similar approach to customize which configuration values you load at runtime by loading different files when running in development versus production. This approach is common; it's included out of the box in most ASP.NET Core templates and by default when you use the default `ConfigurationManager` included with `WebApplicationBuilder`.

10.4.2 Loading environment-specific configuration files

The `EnvironmentName` value is determined early in the process of bootstrapping your application, before the default `ConfigurationManager` is fully populated by `WebApplicationBuilder`. As a result, you can dynamically change which configuration providers are added to the builder and, hence, which configuration values are loaded when the `IConfiguration` is built.

A common pattern is to have an optional, environment-specific appsettings .ENVIRONMENT.json file that's loaded after the default appsettings.json file. The following listing shows how you could achieve this task if you're customizing the `ConfigurationManager` in Program.cs, but it's also effectively what `WebApplication-Builder` does by default.

Listing 10.13 Adding environment-specific appsettings.json files

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

IHostEnvironment env = builder.Environment;          ❶

builder.Configuration.Sources.Clear();
builder.Configuration
    .AddJsonFile(
        "appsettings.json",
        optional: false)                             ❷
    .AddJsonFile(                                    ❸
        $"appsettings.{env.EnvironmentName}.json",   ❸
        Optional: true);                             ❸
```